

EXPERIMENT – 8

Title

To Make Use of Transaction Control Statements viz. COMMIT, ROLLBACK and SAVEPOINT

Aim / Objective

To understand and implement **Transaction Control Language (TCL)** commands such as **COMMIT**, **ROLLBACK**, and **SAVEPOINT** for managing transactions and maintaining database consistency.

Theory

A **Transaction** is a sequence of one or more SQL statements executed as a single logical unit of work. Transaction Control Language (TCL) commands are used to manage changes made by Data Manipulation Language (DML) statements such as **INSERT**, **UPDATE**, and **DELETE**.

A transaction follows the **ACID** properties:

- **Atomicity** – Either all operations are completed or none are.
- **Consistency** – Maintains database consistency before and after a transaction.
- **Isolation** – Ensures transactions execute independently.
- **Durability** – Once committed, changes become permanent.

The commonly used TCL commands are:

- **COMMIT**
- **ROLLBACK**
- **SAVEPOINT**

Transaction Control Statements

1. COMMIT

The **COMMIT** statement permanently saves all changes made during the current transaction.

2. ROLLBACK

The **ROLLBACK** statement undoes all changes made since the last **COMMIT** or rolls back to a specified **SAVEPOINT**.

3. SAVEPOINT

The **SAVEPOINT** statement creates a checkpoint within a transaction, allowing partial rollback.

Learning Objectives

After completing this experiment, students will be able to:

- Understand database transactions.
- Use COMMIT to permanently save changes.
- Use ROLLBACK to undo changes.
- Create SAVEPOINTS for partial rollback.
- Manage database consistency using TCL commands.

Software Requirements

- Oracle Database
- Oracle SQL Developer / SQL*Plus

Algorithm / Procedure

1. Open Oracle SQL Developer.
2. Connect to the Oracle database.
3. Create the Employee table.
4. Insert sample records.
5. Perform UPDATE operations.
6. Apply COMMIT, ROLLBACK, and SAVEPOINT commands.
7. Display the results.
8. Stop.

Program

Step 1 : Create Employee Table

```
CREATE TABLE Employee  
(
```

```
EmpID NUMBER PRIMARY KEY,  
EmpName VARCHAR2(30),  
Department VARCHAR2(30),  
Salary NUMBER(10,2)  
);
```

Step 2 : Insert Sample Records

```
INSERT INTO Employee VALUES(101,'Rahul','IT',35000);  
  
INSERT INTO Employee VALUES(102,'Priya','HR',42000);  
  
INSERT INTO Employee VALUES(103,'Aman','Finance',39000);  
  
COMMIT;
```

Step 3 : Display Records

```
SELECT * FROM Employee;
```

Expected Output

EmpID	EmpName	Department	Salary
101	Rahul	IT	35000
102	Priya	HR	42000
103	Aman	Finance	39000

Program 1 : COMMIT Statement

```
UPDATE Employee  
  
SET Salary=45000  
  
WHERE EmpID=101;  
  
COMMIT;
```

Verify

```
SELECT * FROM Employee;
```

Expected Output

EmpID	EmpName	Salary
-------	---------	--------

101	Rahul	45000
-----	-------	-------

102	Priya	42000
-----	-------	-------

103	Aman	39000
-----	------	-------

Program 2 : ROLLBACK Statement

```
UPDATE Employee
```

```
SET Salary=50000
```

```
WHERE EmpID=102;
```

```
ROLLBACK;
```

Verify

```
SELECT * FROM Employee;
```

Expected Output

The salary of Employee **102** remains unchanged because the transaction was rolled back.

EmpID	EmpName	Salary
-------	---------	--------

101	Rahul	45000
-----	-------	-------

102	Priya	42000
-----	-------	-------

103	Aman	39000
-----	------	-------

Program 3 : SAVEPOINT Statement

```
UPDATE Employee
```

```
SET Salary=48000
```

```
WHERE EmpID=101;
```

```
SAVEPOINT SP1;
```

```
UPDATE Employee
SET Salary=52000
WHERE EmpID=103;
ROLLBACK TO SP1;
COMMIT;
```

Verify

```
SELECT * FROM Employee;
```

Expected Output

EmpID	EmpName	Salary
101	Rahul	48000
102	Priya	42000
103	Aman	39000

Program 4 : Multiple SAVEPOINTS

```
UPDATE Employee
SET Salary=46000
WHERE EmpID=101;
SAVEPOINT A;
UPDATE Employee
SET Salary=44000
WHERE EmpID=102;
SAVEPOINT B;
UPDATE Employee
SET Salary=43000
WHERE EmpID=103;
```

ROLLBACK TO B;

COMMIT;

Verify

SELECT * FROM Employee;

Expected Output

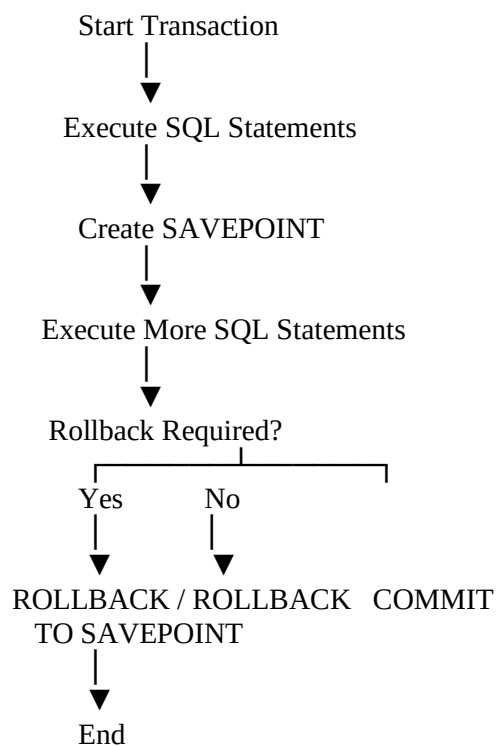
EmpID Salary

101 46000

102 44000

103 39000

Transaction Flow



Explanation

COMMIT

Permanently saves all changes made during the current transaction.

ROLLBACK

Cancels all changes made since the last COMMIT.

SAVEPOINT

Creates a checkpoint inside a transaction.

ROLLBACK TO SAVEPOINT

Restores the transaction to the specified savepoint without affecting earlier committed changes.

Observation

- Employee records were updated successfully.
- COMMIT permanently saved the changes.
- ROLLBACK successfully canceled uncommitted changes.
- SAVEPOINT enabled partial rollback within a transaction.
- Database consistency was maintained throughout the experiment.

Result

Transaction Control Language commands **COMMIT**, **ROLLBACK**, and **SAVEPOINT** were successfully implemented. The experiment demonstrated how transactions can be managed to ensure data consistency, integrity, and recovery in a database.

Applications

- Banking Transactions
- ATM Systems
- Online Shopping Applications

- Railway Reservation Systems
- Hospital Management Systems
- Inventory Management Systems
- Payroll Applications

Advantages

- Ensures data consistency.
- Prevents accidental data loss.
- Supports transaction recovery.
- Enables partial rollback.
- Improves database reliability.

Viva Questions

1. What is a transaction?
2. What are TCL commands?
3. What is the purpose of the COMMIT statement?
4. What is the difference between COMMIT and ROLLBACK?
5. What is a SAVEPOINT?
6. Can we rollback after a COMMIT?
7. What is the purpose of ROLLBACK TO SAVEPOINT?
8. What are ACID properties?
9. Which SQL statements start a transaction?
10. Why are transaction control statements important?

Lab Assignment

1. Create an Employee table and insert five records.
2. Update employee salaries and use COMMIT.
3. Delete a record and use ROLLBACK.

4. Demonstrate multiple SAVEPOINTS in a transaction.
5. Perform partial rollback using SAVEPOINT.
6. Compare COMMIT and ROLLBACK with examples.
7. Explain ACID properties with suitable examples.
8. Create a banking transaction scenario using TCL commands.
9. Demonstrate transaction management using INSERT, UPDATE, and DELETE.
10. Explain the importance of transaction control in database applications.

Precautions

- Always verify records before executing COMMIT.
- Use SAVEPOINT before performing critical database updates.
- Remember that ROLLBACK cannot undo changes after a COMMIT.
- Test transaction commands on sample data before applying them to production databases.
- Verify the output using the SELECT statement after each transaction.

Learning Outcome

After completing this experiment, students will be able to:

- Understand the concept of database transactions.
- Use COMMIT to permanently save changes.
- Apply ROLLBACK to undo unwanted changes.
- Create and use SAVEPOINTS for partial transaction recovery.
- Manage database consistency and reliability using Transaction Control Language (TCL) commands.
- Apply transaction management concepts in real-world applications such as banking, inventory, and reservation systems.